Estudo de Caso 03: Comparação de desempenho de duas configurações de um algoritmo de otimização

Beatriz S. Campagnaro, Filipe C. Andrade e Izabela C. S. Barbosa

12 de Outubro de 2026

Resumo

Este estudo compara o desempenho de duas configurações do algoritmo de evolução diferencial (DE) aplicadas à classe de funções Rosenbrock com dimensão entre 2 e 150. O delineamento adotado é um experimento em blocos completos casualizados (RCBD), em que cada instância (dimensão) constitui um bloco e as duas configurações são os tratamentos. A variável resposta é a média do melhor valor da função objetivo (F_{best}) sobre as repetições em cada célula, transformada pela função logaritmo para estabilizar a escala entre instâncias. O teste principal é o teste t pareado sobre as diferenças dos logaritmos das médias de bloco, equivalente ao teste F do RCBD quando há dois tratamentos. Os resultados da análise, lorem lroem lorem.

Introdução

Algoritmos baseados em populações constituem uma alternativa comum para a solução de problemas de otimização (Storn e Price, 1997). A evolução diferencial (DE) se destaca pela flexibilidade na combinação de operadores de mutação e recombinação (Campelo e Botelho, 2016). Este estudo compara duas configurações do DE aplicadas à classe de funções Rosenbrock (Rosenbrock, 1960), implementada pelo pacote `smoof` (Bossek, 2017), com dimensão $\dim \in [2, 150]$. O critério de desempenho é o melhor valor da função objetivo (F_{best}), em que valores menores indicam melhor desempenho. O objetivo é responder se existe diferença no desempenho médio entre as duas configurações e, caso exista, quantificá-la.

Delineamento Experimental

O delineamento adotado é um RCBD. Cada instância do problema (uma dimensão sorteada em $[2, 150]$) constitui um bloco, e as duas configurações são os tratamentos:

- **Configuração 1:** recombinação exponencial (`recombination_exp`, $cr = 0{,}6$) com mutação baseada no melhor indivíduo (`mutation_best`, $f = 2$).
- **Configuração 2:** recombinação geométrica (`recombination_geo`, $\alpha = 0{,}6$) com mutação aleatória (`mutation_rand`, $f = 1{,}2$).

Os demais parâmetros são comuns a ambas: seleção padrão (`selection_standard`), critério de parada por número de avaliações ($5000 \cdot \dim$), tamanho da população ($5 \cdot \dim$), e domínio $[-5, 10]^{\dim}$. Cada bloco contém exatamente uma observação por tratamento, correspondente à média de F_{best} sobre as repetições; tratar cada repetição individual como observação independente constituiria pseudorreplicação (Campelo e Takahashi, 2019). As instâncias são sorteadas sem reposição com semente fixa, e a ordem de execução das células é aleatorizada.

Formulação das hipóteses

O valor de F_{best} cresce em várias ordens de grandeza com a dimensão. Para tornar as comparações coerentes, a análise é conduzida na escala logarítmica. Seja $\bar{Y}_{ij}$ a média de F_{best} sobre as repetições da

configuração i na instância j. A variável analisada é a diferença pareada:

$$D_j = \log \bar{Y}_{1j} - \log \bar{Y}_{2j} = \log \frac{\bar{Y}_{1j}}{\bar{Y}_{2j}}$$

Se $D_j > 0$, a Configuração 2 apresenta valor menor de F_{best} (melhor desempenho) na instância j. As hipóteses são:

$$\begin{cases} H_0 : \mu_D = 0 \\ H_1 : \mu_D \neq 0 \end{cases}$$

em que μ_D é a média populacional das diferenças pareadas na escala logarítmica. O teste é bilateral. O efeito padronizado minimamente relevante é $d^* = \delta^*/\sigma_\Phi = 0{,}5$ (Campelo e Takahashi, 2019) e o nível de significância é $\alpha = 0{,}05$. Com dois tratamentos ($a = 2$), o teste F do RCBD é equivalente ao quadrado da estatística t do teste t pareado (Montgomery, 2005, Cap. 4).

Cálculo do tamanho amostral

O número de instâncias N é determinado pela exigência de potência $1 - \beta = 0{,}8$ no teste t pareado bilateral. Como $d^* = 0{,}5$ já é normalizado por σ_Φ, o cálculo independe de estimativas de variância.

```
# Cálculo iterativo do N mínimo para potência >= 0.8
calc_power_paired <- function(N, d_star = 0.5, alpha = 0.05) {
  df     <- N - 1
  ncp    <- d_star * sqrt(N)
  t_crit <- qt(1 - alpha/2, df)
  power  <- 1 - pt(t_crit, df, ncp) + pt(-t_crit, df, ncp)
  return(power)
}

N_range <- 30:40
power_table <- data.frame(
  N     = N_range,
  Poder = sapply(N_range, calc_power_paired)
)

knitr::kable(
  power_table,
  caption = "\\label{tab:poder_N}Potência do teste $t$ pareado bilateral em função de $N$, com $dˆ* = 0
  digits = 3,
  align = "c",
  booktabs = TRUE
)
```

Tabela 1: Potência do teste t pareado bilateral em função de N, com $d^* = 0{,}5$ e $\alpha = 0{,}05$

N	Poder
30	0.754
31	0.768
32	0.782
33	0.795
34	0.808

N	Poder
35	0.820
36	0.831
37	0.841
38	0.851
39	0.861
40	0.869

```
N <- min(power_table$N[power_table$Poder >= 0.8])
poder_N <- calc_power_paired(N)
```

A Tabela 1 mostra que $N = 34$ é o menor valor que atinge potência superior a 0,80 ($\pi = 0.808$).

O número de repetições n por célula controla a precisão da estimativa $\bar{Y}_{ij}$ e é definido pelo critério de Campelo e Takahashi (2019): $\text{se}^{*2} \leq 0{,}1 \cdot \hat{\sigma}^2_\Phi$. Para estimar os parâmetros necessários, um estudo piloto é conduzido.

```
set.seed(2026)
pilot_dims <- sort(sample(2:150, 5))
n_pilot    <- 15
```

```
pilot_data <- read.csv("dados_piloto_EC03.csv", stringsAsFactors = FALSE)
```

```
pilot_sds <- pilot_data %>%
  group_by(dim, config) %>%
  summarise(sd_log = sd(log(Fbest)), .groups = "drop") %>%
  pivot_wider(names_from = config, values_from = sd_log,
              names_prefix = "sd_")

knitr::kable(
  pilot_sds,
  caption = "\\label{tab:pilot_sd}Desvio padrão de $\\log(F_{\\text{best}})$ por instância e configuraç
  digits = 4,
  col.names = c("Dimensão", "$\\hat{\\sigma}_1$", "$\\hat{\\sigma}_2$"),
  align = "c",
  booktabs = TRUE,
  escape = FALSE
)
```

Tabela 2: Desvio padrão de $\log(F_{\text{best}})$ por instância e configuração no estudo piloto

Dimensão	$\hat{\sigma}_1$	$\hat{\sigma}_2$
39	0.1090	0.2351
46	0.1588	0.2593
92	0.1160	0.1655
112	0.1304	0.1433
122	0.1009	0.1516

```
s1_pool <- sqrt(mean(pilot_sds$sd_Config1^2))
s2_pool <- sqrt(mean(pilot_sds$sd_Config2^2))

pilot_block <- pilot_data %>%
```

```
  group_by(dim, config) %>%
  summarise(mean_Fbest = mean(Fbest), .groups = "drop") %>%
  pivot_wider(names_from = config, values_from = mean_Fbest)

pilot_block$log_diff <- log(pilot_block$Config1) - log(pilot_block$Config2)
sigma_phi_pilot <- sd(pilot_block$log_diff)

target_se2 <- 0.1 * sigma_phi_pilot^2

n1_opt <- ceiling(s1_pool * (s1_pool + s2_pool) / target_se2)
n2_opt <- ceiling(n1_opt * s2_pool / s1_pool)
n_equal <- ceiling((s1_pool^2 + s2_pool^2) / target_se2)
```

O piloto utilizou 5 dimensões (39, 46, 92, 112, 122), 15 repetições por célula. As estimativas combinadas são $\hat{\sigma}_1 = 0.1247$ e $\hat{\sigma}_2 = 0.1967$, e $\hat{\sigma}_\Phi = 1.7603$. O critério $\text{se}^{*2} \leq 0.3099$ resulta em $n = 1$ com alocação igual.

```
n_final <- max(n_equal, 30)
```

```
if (n_equal <= 30) {
  cat(sprintf(
    "\\qquad O valor calculado ($n = %d$) é inferior a 30. Adota-se $n = 30$ repetições por configuraçã
    n_equal, (s1_pool^2 + s2_pool^2)/30, target_se2))
} else {
  cat(sprintf(
    "\\qquad O valor calculado ($n = %d$) é adotado. A verificação confirma: $\\text{se}^{*2} = %.4f \\l
    n_equal, (s1_pool^2 + s2_pool^2)/n_equal, target_se2))
}
```

O valor calculado ($n = 1$) é inferior a 30. Adota-se $n = 30$ repetições por configuração por instância como margem conservadora (Campelo, 2018). A verificação confirma que o critério é atendido: $\text{se}^{*2} = 0.0018 \leq 0.3099$.

Coleta dos dados

O experimento utiliza $N = 34$ instâncias e $n = 30$ repetições por configuração por instância. As dimensões são amostradas sem reposição de $\{2, \ldots, 150\}$, excluindo as do piloto, com semente fixa.

```
dados_brutos <- read.csv("dados_EC03_grupoB.csv", stringsAsFactors = FALSE)
```

```
n_obs <- nrow(dados_brutos)
n_config1 <- sum(dados_brutos$Algorithm == "Config1")
n_config2 <- sum(dados_brutos$Algorithm == "Config2")
n_instances <- length(unique(dados_brutos$Instance))

tabela_coleta <- data.frame(
  Item = c("Total de observações",
           "Observações Config 1",
           "Observações Config 2",
           "Número de instâncias",
           "Repetições por célula"),
  Valor = c(n_obs, n_config1, n_config2, n_instances, n_final),
  check.names = FALSE
)

knitr::kable(
  tabela_coleta,
```

```
  caption = "\\label{tab:coleta}Resumo da coleta de dados",
  align = c("l", "c"),
  booktabs = TRUE
)
```

Tabela 3: Resumo da coleta de dados

Item	Valor
Total de observações	2040
Observações Config 1	1020
Observações Config 2	1020
Número de instâncias	34
Repetições por célula	30

A Tabela 3 confirma a integridade da coleta.

Análise Exploratória

```
block_means <- dados_brutos %>%
  group_by(Algorithm, Instance) %>%
  summarise(mean_Fbest = mean(Result), .groups = "drop")

block_wide <- block_means %>%
  pivot_wider(names_from = Algorithm, values_from = mean_Fbest) %>%
  mutate(log_C1   = log(Config1),
         log_C2   = log(Config2),
         D        = log_C1 - log_C2)

D_bar    <- mean(block_wide$D)
S_D      <- sd(block_wide$D)
N_used   <- nrow(block_wide)
d_obs    <- D_bar / S_D
```

```
block_long <- block_wide %>%
  select(Instance, log_C1, log_C2) %>%
  pivot_longer(cols = c(log_C1, log_C2),
               names_to = "Config",
               values_to = "log_mean") %>%
  mutate(Config = ifelse(Config == "log_C1", "Config 1", "Config 2"))

ggplot(block_long, aes(x = Config, y = log_mean, fill = Config)) +
  geom_boxplot(alpha = 0.6, outlier.shape = NA) +
  geom_jitter(width = 0.15, size = 1.5, alpha = 0.6) +
  labs(x = "Configuração", y = expression(log(bar(Y)[ij]))) +
  theme_minimal() +
  theme(legend.position = "none")
```

```
ggplot(block_wide, aes(x = D)) +
  geom_histogram(bins = 12, fill = "steelblue", color = "black", alpha = 0.7) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "red") +
  geom_vline(xintercept = D_bar, linetype = "solid", color = "darkblue") +
  labs(x = expression(D[j] == log(bar(Y)["1j"]) - log(bar(Y)["2j"])),
```

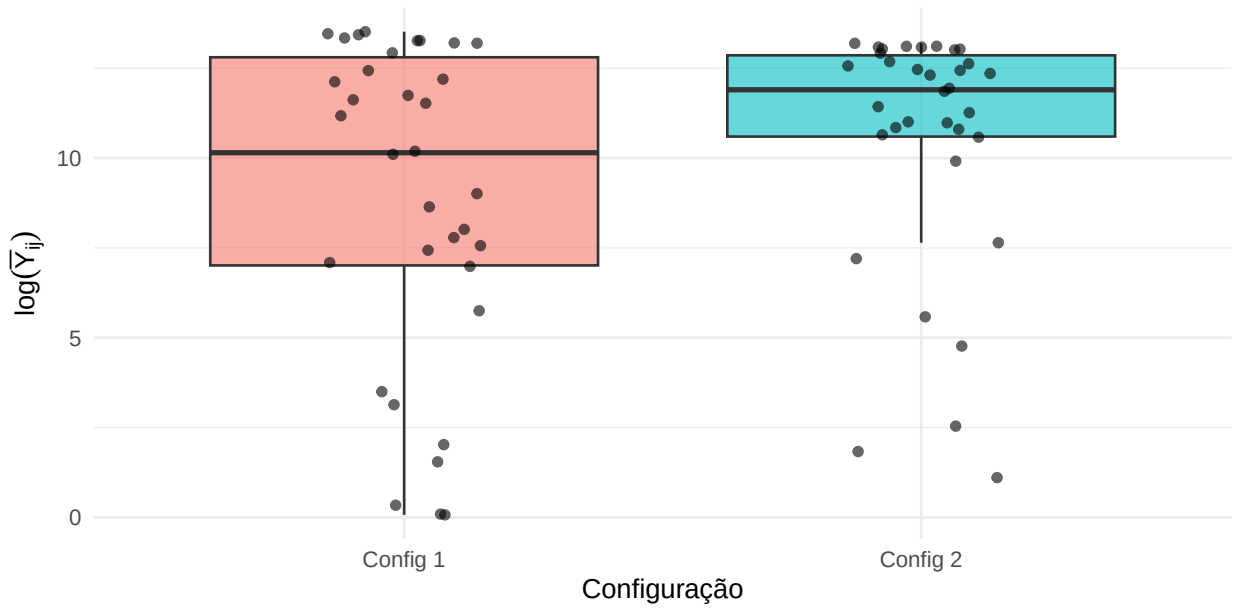


Figura 1: Distribuição de $\log(\bar{Y}_{ij})$ por configuração

```
       y = "Frequência") +
  theme_minimal()
```

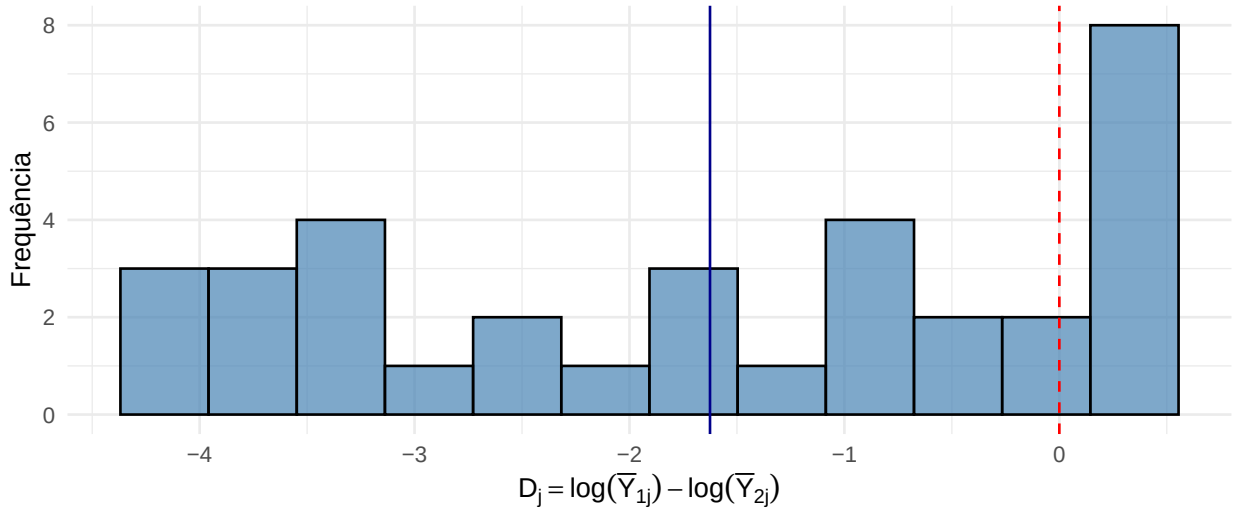


Figura 2: Distribuição das diferenças pareadas D_j

```
block_perfil <- block_wide %>%
  select(Instance, log_C1, log_C2) %>%
  arrange(Instance) %>%
  mutate(idx = row_number())

ggplot() +
  geom_point(data = block_perfil, aes(x = idx, y = log_C1, color = "Config 1")) +
```

```
  geom_point(data = block_perfil, aes(x = idx, y = log_C2, color = "Config 2")) +
  geom_segment(data = block_perfil,
               aes(x = idx, xend = idx, y = log_C1, yend = log_C2),
               alpha = 0.3) +
  scale_color_manual(values = c("Config 1" = "steelblue", "Config 2" = "tomato")) +
  labs(x = "Instância (ordenada por dimensão)", y = expression(log(bar(Y)[ij])),
       color = "Configuração") +
  theme_minimal()
```

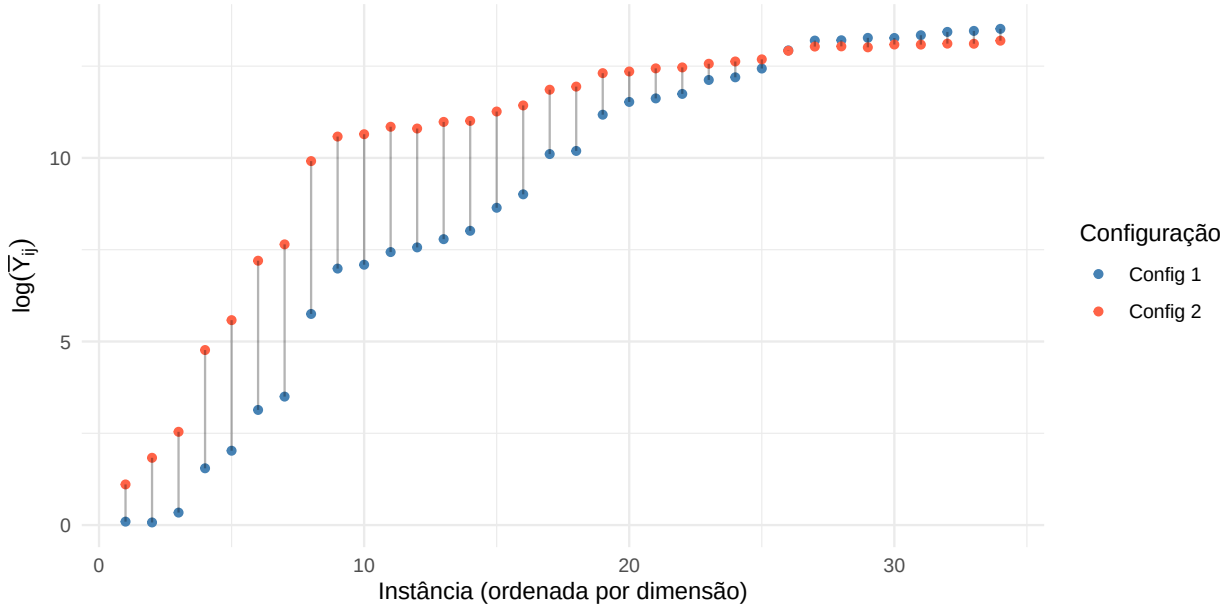


Figura 3: Perfil de desempenho por instância: $\log(\bar{Y}_{ij})$ para cada configuração

Análise Estatística

Teste de hipóteses

O teste principal é o teste t pareado bilateral sobre as diferenças D_j, com $N - 1 = 33$ graus de liberdade. Complementarmente, apresenta-se o teste F do RCBD e verifica-se a equivalência $F = t^2$.

```
teste_t <- t.test(block_wide$D, mu = 0, alternative = "two.sided",
                  conf.level = 0.95)

tabela_t <- data.frame(
  Estatística = c("$\\bar{D}$", "$S_D$", "$t_0$", "g.l.", "p-valor",
                  "IC 95\\% inferior", "IC 95\\% superior"),
  Valor = c(
    sprintf("%.4f", D_bar),
    sprintf("%.4f", S_D),
    sprintf("%.4f", teste_t$statistic),
    N_used - 1,
    format.pval(teste_t$p.value, digits = 4),
    sprintf("%.4f", teste_t$conf.int[1]),
    sprintf("%.4f", teste_t$conf.int[2])
  ),
```

```
  check.names = FALSE
)

knitr::kable(
  tabela_t,
  caption = "\\label{tab:teste_t}Resultado do teste $t$ pareado bilateral",
  align = c("l", "c"),
  booktabs = TRUE,
  escape = FALSE
)
```

Tabela 4: Resultado do teste t pareado bilateral

Estatística	Valor
$\bar{D}$	-1.6254
S_D	1.5870
t_0	-5.9720
g.l.	33
p-valor	1.048e-06
IC 95% inferior	-2.1791
IC 95% superior	-1.0717

```
dados_rcbd <- block_means %>%
  mutate(log_Y     = log(mean_Fbest),
         Instance = factor(Instance),
         Algorithm = factor(Algorithm))

modelo_rcbd <- aov(log_Y ~ Algorithm + Instance, data = dados_rcbd)
anova_rcbd  <- summary(modelo_rcbd)

F_val <- anova_rcbd[[1]]["Algorithm", "F value"]
t_val <- teste_t$statistic
```

O teste F do RCBD para o fator Algorithm resulta em $F = 35.6647$, e $t^2 = 35.6647$, confirmando a equivalência algébrica (Montgomery, 2005).

Estimação da magnitude do efeito

```
d_obs_val <- D_bar / S_D
ratio_geom <- exp(D_bar)
ratio_CI   <- exp(teste_t$conf.int)
delta_star <- 0.5 * S_D

tabela_mag <- data.frame(
  Medida = c(
    "Efeito padronizado ($d = \\bar{D}/S_D$)",
    "$d^*$ (referência)",
    "$\\bar{D}$ (escala log)",
    "$\\delta^* = 0{,}5 \\cdot S_D$ (escala log)",
    "Razão Config 1 / Config 2 ($e^{\\bar{D}}$)",
    "IC 95\\% da razão (inferior)",
    "IC 95\\% da razão (superior)"
```

```
  ),
  Valor = c(
    sprintf("%.4f", d_obs_val),
    "0.5",
    sprintf("%.4f", D_bar),
    sprintf("%.4f", delta_star),
    sprintf("%.4f", ratio_geom),
    sprintf("%.4f", ratio_CI[1]),
    sprintf("%.4f", ratio_CI[2])
  ),
  check.names = FALSE
)

knitr::kable(
  tabela_mag,
  caption = "\\label{tab:magnitude}Estimativas da magnitude do efeito",
  align = c("l", "c"),
  booktabs = TRUE,
  escape = FALSE
)
```

Tabela 5: Estimativas da magnitude do efeito

Medida	Valor
Efeito padronizado ($d = \bar{D}/S_D$)	-1.0242
d^* (referência)	0.5
$\bar{D}$ (escala log)	-1.6254
$\delta^* = 0{,}5 \cdot S_D$ (escala log)	0.7935
Razão Config 1 / Config 2 ($e^{\bar{D}}$)	0.1968
IC 95% da razão (inferior)	0.1131
IC 95% da razão (superior)	0.3424

Análise de sensibilidade

```
d_detect <- NA
for (d_try in seq(0.01, 2, by = 0.01)) {
  pw <- calc_power_paired(N_used, d_star = d_try, alpha = 0.05)
  if (pw >= 0.8) {
    d_detect <- d_try
    break
  }
}
```

Com $N = 34$ instâncias e $\alpha = 0{,}05$, o menor efeito padronizado detectável com potência de 80% é $d = 0.5$. O efeito padronizado observado é $|d| = 1.024$.

Eficiência da blocagem

```
b_blocks  <- N_used
a_treat   <- 2
MSBlocks  <- anova_rcbd[[1]]["Instance", "Mean Sq"]
MSE       <- anova_rcbd[[1]]["Residuals", "Mean Sq"]
```

```
E_block <- ((b_blocks - 1) * MSBlocks + b_blocks * (a_treat - 1) * MSE) /
           ((a_treat * b_blocks - 1) * MSE)
```

A eficiência relativa da blocagem é $E = 12.46$ (Montgomery, 2005). Um CRD precisaria de aproximadamente 1146% mais observações para atingir o mesmo poder.

Verificação das premissas

A regra decisória foi definida antes da coleta: se o teste de Shapiro-Wilk sobre D_j resultar em $p < 0{,}05$, reporta-se o teste de Wilcoxon pareado como resultado principal.

Normalidade das diferenças pareadas:

```
shapiro_D <- shapiro.test(block_wide$D)

car::qqPlot(block_wide$D, pch = 20, main = "", ylab = "Quantis observados",
            xlab = "Quantis teóricos (Normal)")
```

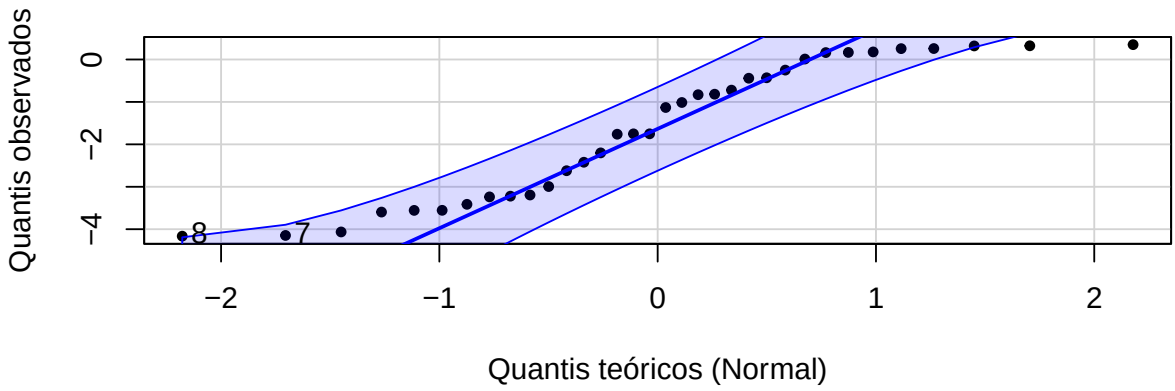


Figura 4: Gráfico Q-Q das diferenças pareadas D_j

```
## [1] 8 7
```

Shapiro-Wilk: $W = 0.8934$, p-valor $= 0.0031$.

Normalidade dos resíduos do RCBD:

```
res_rcbd   <- residuals(modelo_rcbd)
shapiro_res <- shapiro.test(res_rcbd)

car::qqPlot(res_rcbd, pch = 20, main = "", ylab = "Quantis observados",
            xlab = "Quantis teóricos (Normal)")
```

```
## [1]  8 42
```

Shapiro-Wilk: $W = 0.9132$, p-valor $= 0.0002$.

Homogeneidade de variâncias:

```
dados_rcbd$res <- residuals(modelo_rcbd)
fligner_res <- fligner.test(res ~ Algorithm, data = dados_rcbd)
```

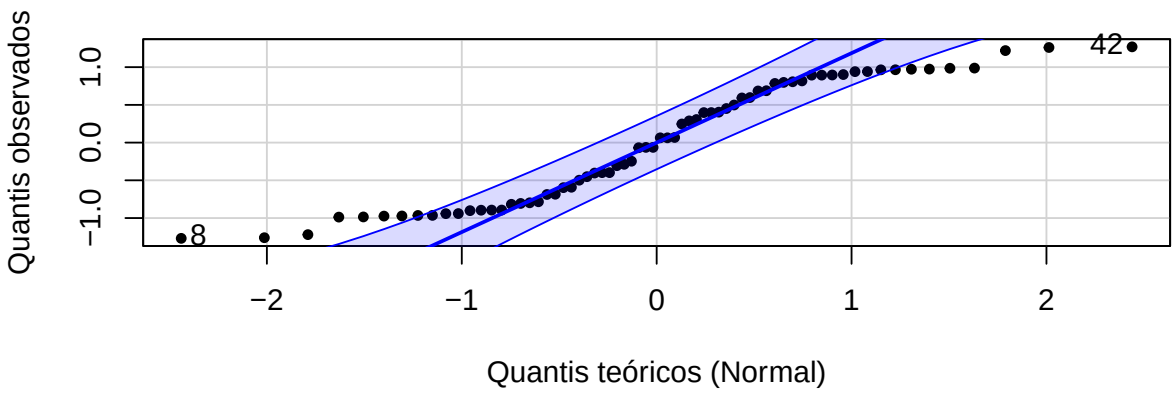


Figura 5: Gráfico Q-Q dos resíduos do modelo RCBD

```
tabela_fligner <- data.frame(
  `Estatística $\\chi^2$` = sprintf("%.4f", fligner_res$statistic),
  `g.l.` = fligner_res$parameter,
  `p-valor` = sprintf("%.4f", fligner_res$p.value),
  check.names = FALSE
)

knitr::kable(
  tabela_fligner,
  caption = "\\label{tab:fligner}Teste de Fligner-Killeen para homogeneidade de variâncias",
  align = "c",
  booktabs = TRUE,
  escape = FALSE
)
```

Tabela 6: Teste de Fligner-Killeen para homogeneidade de variâncias

	Estatística χ^2	g.l.	p-valor
df	0.0005	1	0.9827

Independência:

A premissa de independência não é verificável por teste estatístico e deve ser garantida na etapa de planejamento (Campelo, 2018). Neste experimento, a independência é razoavelmente assumida pelas seguintes razões: (i) a variável resposta é F_{best}, determinada pela qualidade da solução encontrada e não por fatores ambientais como carga computacional; (ii) cada instância corresponde a uma dimensão distinta, constituindo um problema de otimização independente; (iii) cada execução utiliza uma semente pseudoaleatória distinta; e (iv) a ordem de execução das células foi aleatorizada.

```
usar_wilcoxon <- shapiro_D$p.value < 0.05

if (usar_wilcoxon) {
```

```r
  wilcox_res <- wilcox.test(block_wide$D, mu = 0, alternative = "two.sided",
                            conf.int = TRUE, conf.level = 0.95)

  cat("\\qquad Como o teste de Shapiro-Wilk rejeitou a normalidade das diferenças pareadas ($p =",
      sprintf("%.4f", shapiro_D$p.value),
      "$), aplica-se o plano B: teste de Wilcoxon pareado.\n\n")

  tabela_wilcox <- data.frame(
    Medida = c("Estatística $V$", "p-valor",
               "Pseudo-mediana", "IC 95\\% inferior", "IC 95\\% superior"),
    Valor = c(
      sprintf("%.1f", wilcox_res$statistic),
      format.pval(wilcox_res$p.value, digits = 4),
      sprintf("%.4f", wilcox_res$estimate),
      sprintf("%.4f", wilcox_res$conf.int[1]),
      sprintf("%.4f", wilcox_res$conf.int[2])
    ),
    check.names = FALSE
  )

  print(knitr::kable(
    tabela_wilcox,
    caption = "\\label{tab:wilcoxon}Resultado do teste de Wilcoxon pareado",
    align = c("l", "c"),
    booktabs = TRUE,
    escape = FALSE
  ))
}
```

Como o teste de Shapiro-Wilk rejeitou a normalidade das diferenças pareadas ($p = 0.0031 $), aplica-se o plano B: teste de Wilcoxon pareado.

Tabela 7: Resultado do teste de Wilcoxon pareado

Medida	Valor
Estatística V	50.0
p-valor	3.604e-06
Pseudo-mediana	-1.6250
IC 95% inferior	-2.1572
IC 95% superior	-1.0009

Conclusões e Recomendações

Declaração de uso de IA generativa e tecnologias assistidas por IA

Durante a preparação deste trabalho, os autores utilizaram o Claude, da Anthropic (modelo Opus 4.6), com o objetivo de auxiliar na revisão técnica do planejamento experimental. Após utilizar esse serviço, os autores revisaram e editaram o conteúdo conforme necessário e assumem total responsabilidade pelo conteúdo do material publicado.

Papéis e Contribuições da Equipe

- **Beatriz S. Campagnaro:** Desenvolvimento dos scripts estatísticos em R, redação do manuscrito e revisão técnica.

- **Filipe C. Andrade:** Desenvolvimento dos scripts estatísticos em R, redação do manuscrito e revisão técnica.
- **Izabela C. S. Barbosa:** Desenvolvimento dos scripts estatísticos em R, redação do manuscrito e revisão técnica.

Referências

CAMPELO, F.; TAKAHASHI, R. H. C. Sample size estimation for power and accuracy in the experimental comparison of algorithms. *Journal of Heuristics*, v. 25, n. 2, p. 305–338, 2019.

CAMPELO, F.; BOTELHO, M. Experimental investigation of recombination operators for differential evolution. In: *Proc. Genetic and Evolutionary Computation Conference (GECCO'2016)*, 2016.

CAMPELO, F. *Lecture Notes on Design and Analysis of Experiments*. Versão 2.12, 2018. Disponível em: http://git.io/v3Kh8.

MONTGOMERY, D. C. *Design and Analysis of Experiments*. 6. ed. Hoboken: John Wiley & Sons, 2005.

MONTGOMERY, D. C.; RUNGER, G. C. *Applied Statistics and Probability for Engineers*. 5. ed. Hoboken: John Wiley & Sons, 2010.

STORN, R.; PRICE, K. Differential evolution: a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, v. 11, n. 4, p. 341–359, 1997.

ROSENBROCK, H. H. An automatic method for finding the greatest or least value of a function. *Computer Journal*, v. 3, n. 3, p. 175–184, 1960.

BOSSEK, J. smoof: Single- and Multi-Objective Optimization Test Functions. *The R Journal*, v. 9, n. 1, p. 103–113, 2017.


```
## sessionInfo():

## R version 4.6.1 (2026-06-24 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26300)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Portuguese_Brazil.utf8  LC_CTYPE=Portuguese_Brazil.utf8
## [3] LC_MONETARY=Portuguese_Brazil.utf8 LC_NUMERIC=C
## [5] LC_TIME=Portuguese_Brazil.utf8
##
## time zone: America/Sao_Paulo
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] kableExtra_1.4.1    knitr_1.51          car_3.1-5
##  [4] carData_3.0-6       tidyr_1.3.2         dplyr_1.2.1
##  [7] ggplot2_4.0.3       smoof_1.7.0         checkmate_2.3.4
## [10] ParamHelpers_1.14.2 ExpDE_0.2.0
##
## loaded via a namespace (and not attached):
##  [1] generics_0.1.4      xml2_1.6.0          stringi_1.8.9
##  [4] digest_0.6.39       magrittr_2.0.5      evaluate_1.0.5
```

```
##  [7] grid_4.6.1          RColorBrewer_1.1-3  BBmisc_1.13.1
## [10] fastmap_1.2.0       backports_1.5.1     Formula_1.2-6
## [13] purrr_1.2.2         viridisLite_0.4.3   scales_1.4.0
## [16] textshaping_1.0.5   abind_1.4-8         cli_3.6.6
## [19] rlang_1.3.0         withr_3.0.3         yaml_2.3.12
## [22] otel_0.2.0          tools_4.6.1         fastmatch_1.1-8
## [25] vctrs_0.7.3         R6_2.6.1            lifecycle_1.0.5
## [28] stringr_1.6.0       pkgconfig_2.0.3     pillar_1.11.1
## [31] gtable_0.3.6        data.table_1.18.6.1 glue_1.8.1
## [34] Rcpp_1.1.2          systemfonts_1.3.2   xfun_0.60
## [37] tibble_3.3.1        tidyselect_1.2.1    rstudioapi_0.19.0
## [40] farver_2.1.2        htmltools_0.5.9     labeling_0.4.3
## [43] rmarkdown_2.31      svglite_2.2.2       compiler_4.6.1
## [46] S7_0.2.2
```